DEVELOPER TOOLS

AI · ML · DS / REVISION NOTES

YOUTUBE

Pipenv Crash Course

Managing Python versions, virtual environments and dependencies with Pipenv

6
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Source: https://www.youtube.com/watch?v=6Qmnh5C4Pmo

---

1. Python version

To check python version in terminal:

```
python --version
```

If you are using Python 3 specifically:

```
python3 --version
```

---

2. Install pipenv

Install pipenv using VSCode terminal:

```
pip3 install pipenv
```

---

3. Check installed packages

To see everything installed in the current global environment:

```
pip3 freeze
```

This shows all packages in the **current virtual/global environment**.

---

4. Create virtual environment

Run:

```
pipenv shell
```

This creates:

- a **Pipfile**
- a **virtual environment**

The **Pipfile** is similar to:

```
package.json
```

in Node.js.

---

5. Python version inside virtual environment

After activating the virtual environment, run:

```
python --version
```

This will show the **Python version of the virtual environment**.

---

6. Exit virtual environment

To exit the virtual environment:

```
exit
```

---

7. Activate virtual environment again

Run again:

```
pipenv shell
```

This activates the virtual environment.

---

8. Install example package

Install the `camelcase` package:

```
pipenv install camelcase
```

Now check the **Pipfile**.

You will see:

```
camelcase = "*"
```

which means the **latest version** of camelcase will be installed.

Example program can be executed using:

```
python example.py
```

---

9. List all installed packages

To list all packages and their dependencies:

```
pipenv graph
```

---

10. Uninstall a package

To uninstall a package:

```
pipenv uninstall camelcase
```

---

11. Install development packages

To install development-only packages (not for production):

```
pipenv install nose --dev
```

Now check the **Pipfile** again.

---

12. Install packages from requirements.txt

If a `requirements.txt` file exists in the directory:

```
pipenv install -r ./requirements.txt
```

Then check the **Pipfile** again.

---

13. Security and vulnerability check

To check for security vulnerabilities:

```
pipenv check
```

or

```
pipenv audit
```

---

14. Update packages after changing Pipfile

If you manually change versions in the Pipfile, run:

```
pipenv install
```

This updates the installed packages accordingly.

---

15. Show dependency graph

To display dependencies between packages:

```
pipenv graph
```

---

16. Deployment on server

For deployment:

Use **Pipfile.lock** instead of Pipfile.

Steps:

First generate lock file:

```
pipenv lock
```

Then install using:

```
pipenv install --ignore-pipfile
```

This ensures exact dependency versions from **Pipfile.lock**.

---

17. Run python using pipenv

Exit the virtual environment first:

```
exit
```

Then run the program using:

```
pipenv run python example.py
```

To verify Python interpreter being used:

```
pipenv run python
```

Inside Python interpreter run:

```
import sys
sys.executable
```

Then exit interpreter:

```
quit()
```

---

18. Find virtual environment location

To see the location of the virtual environment:

```
pipenv --venv
```

If the location is:

```
/Users/ankit/.local/share/virtualenvs/
```

You can delete all virtual environments using:

```
rm -rf /Users/ankit/.local/share/virtualenvs/*
```

---

19. Run tests

To run tests using pipenv:

```
pipenv run pytest .
```